FROSTBIT: Proof-Carrying Execution Plans for Boolean Filtering over Compressed Bitmaps

William Liu
Exa
San Francisco, USA
wliu@exa.ai

ABSTRACT

Boolean filtering over Roaring-format compressed bitmaps is a core primitive of search and analytics systems. Prior work optimizes such queries dynamically: cost models select per-node algorithms and representations at evaluation time, buffers are allocated as results materialize, and pruning decisions are made while operands are consumed. This paper develops the static alternative for the read-only setting in which serving systems actually operate — immutable index segments and filters compiled once, evaluated many times. FROSTBIT compiles a boolean expression into a *proof-carrying execution plan*: a bottom-up analysis derives every operator's output-container set together with a byte ceiling per container that is sound for every intermediate state of the fold, from which three guarantees follow at plan time rather than run time — execution is allocation-free by construction with a closed-form working-memory bound; a container-granular liveness mask prunes, before any input byte is read, every 64 Ki-value block that cannot contribute to the result of the whole expression; and each operator's fold order is selected by comparing its proven accumulator footprint to the cache hierarchy. Representation and kernel choices are resolved by cost models over plan-derived quantities, including a fan-in-aware array-to-bitmap promotion rule that dominates, under the calibrated model, the fixed cardinality thresholds used in deployed Roaring implementations.

Against both configurations of the standard Rust Roaring library, with all n-ary baseline calls routed through the library's documented `MultiOps` fast path, FROSTBIT wins 32 of a 2–16-way operation sweep's 36 cells outright — every sorted-array cell (1.2–12.2×), every intersection and union cell — and ties the remaining four (small difference cells, 0.94–1.00×, inside measurement noise), with no losses. End-to-end it evaluates a 25,000-tree filter corpus 2.7× and 2.2× faster, with per-mechanism ablations — including rejected designs — isolating the contribution of each planning decision. Against the production substrate it succeeds at a commercial web-scale search engine — an equally-tuned engine of the same lineage that re-derives its decisions during each evaluation — FROSTBIT wins 34 of 36 operation cells and 3–8% end-to-end, with tail shapes improving up to 31×: the measured value of carrying decisions in the plan rather than re-deriving them.

1 INTRODUCTION

Serving systems evaluate boolean filter predicates — conjunctions, disjunctions, and negations over per-attribute document sets — on the critical path of every query. The dominant physical representation is the Roaring bitmap [43, 54]: a two-level structure whose 64 Ki-value *containers* adaptively select among sorted arrays, packed bitmaps, and run encodings, and whose per-container kernels have been extensively optimized with SIMD techniques [34, 39, 46, 55, 63].

Optimizing the *expression* above the containers is a problem with a substantial history. Johnson [13] measured the large algorithm-choice space for compressed-bitmap operations; Amer-Yahia and Johnson [14] built a dynamic-programming optimizer that selects per-node algorithms and intermediate formats for boolean expression trees over RLE-compressed bitmaps; Kaser and Lemire [45] fitted cost formulas to choose evaluation strategies for multi-operand queries over word-aligned (EWAH) bitmaps; Kim et al. [56] develop calibrated cost models for per-query intersection-algorithm selection in web search; and a theoretical line establishes worst-case-optimal evaluation of union-intersection expressions [20, 23]. These optimizers decide either immediately before each evaluation, from estimated operand statistics [14, 45, 56], or during it, adapting to observed selectivities [24]; in both cases the execution machinery beneath them allocates and sizes intermediate results as they materialize, and none emits guarantees about the execution it has chosen.

This paper examines the opposite point in the design space, motivated by the regime in which filtering actually runs in production: index segments are immutable artifacts of an indexing pipeline, so every statistic a plan depends on — each leaf's exact per-container key set, cardinality, and representation — is exact, not estimated, and can never be invalidated. In this setting adaptivity has nothing to adapt to, and we show that the entire optimization surface can be discharged *statically* at plan construction (whose cost is included in every measurement we report), yielding plans that carry guarantees no estimate-driven or adaptive optimizer provides:

- **Allocation-freeness as a plan-time invariant.** The analysis derives, for every operator, its exact output-container key set and a per-container byte ceiling (*capacity clamp*) sound for every intermediate state of the fold (§4.2). Consequently the execution path contains no allocation or growth code, working memory is bounded in closed form (§4.3), and results serialize in place inside the working arena (§4.4). Engines such as Trill [42] and Photon [65] pool and bound allocations operationally; CRoaring pre-sizes individual operations by cardinality analysis [43]. FROSTBIT differs in deriving the bound for the *whole expression* from the plan, and in the stronger consequence that the steady-state allocation count is exactly zero, a property we verify empirically.
- **Static, expression-wide, container-granular pruning.** An upward pass computes each node's surviving key set; when the root's set is narrower than a child's, the plan carries a liveness mask and every leaf cursor skips dead

containers before they are read (§7.1). Zone maps prune scans against per-block metadata [11, 49]; skip pointers and block-max structures prune *dynamically* during list traversal [7, 33]; the Java Roaring library's `workShyAnd` computes a key mask at run time for flat n-ary intersection [71]; partition-level prefilters prune pairwise [63, 68]. FROSTBIT generalizes these to a *static* mask over arbitrary AND/OR/DIFF trees, derived from the plan itself, with a soundness argument (Lemma 1) and with no per-evaluation decision cost.

- **Fold order selected from a proven footprint.** n-ary folds admit two orders — complete each key across all operands, or stream each operand across all keys — which are the container-algebra analogues of document-at-a-time and term-at-a-time evaluation [6], and which we show are memory-optimal in disjoint regimes on modern hierarchies. Cache-conscious cost models for operator layout are classical [3, 18, 19]; FROSTBIT's distinction is that the discriminating quantity — the accumulator footprint — is not an estimate but the sum of proven clamps, so the decision inherits the plan's soundness (§6).

Beneath the plan, representation and kernel selection are governed by explicit cost models over plan-derived quantities (§5); the kernels themselves synthesize the published state of the art [34, 39, 46, 64] and are measured at parity with the baseline's, so that end-to-end differences are attributable primarily to the planning layer rather than kernel quality (§9).

Contributions. C1. A compilation scheme from boolean container-algebra expressions to proof-carrying plans[1]: sound output shapes, per-container capacity clamps with a case-analytic soundness argument, and a closed-form working-memory bound; allocation-free execution and in-place result serialization follow as invariants (§4, §8). **C2.** A static, container-granular liveness analysis for arbitrary AND/OR/DIFF trees with a soundness proof and a streamed-bytes account, automatically enabled when the plan proves it profitable (§7.1). **C3.** Plan-time algorithm selection with derived break-evens: a fan-in-aware array-to-bitmap promotion rule that dominates fixed cardinality thresholds, and footprint-gated fold-order selection, including the control experiments that scoped where each applies (§5, §6). **C4.** An evaluation against both configurations of the Rust Roaring library and against the production substrate FROSTBIT succeeds, with complete per-mechanism ablations including negative results, a 25,000-tree corpus, and a per-tree audit methodology; FROSTBIT is released as open source (§9, §10).

2 RELATED WORK

Expression-level optimization of bitmap and set queries. The closest line to this work begins with performance studies of compressed-bitmap operation choices [13] and the Amer-Yahia–Johnson optimizer [14], which plans boolean expression trees over RLE bitmaps by dynamic programming over per-node algorithm and format choices under density-based cost models. Kaser and Lemire [45] extend cost-based strategy selection to multi-operand queries over EWAH-compressed bitmaps (including heuristics for intermediate materialization), and Kim et al. [56] build and validate calibrated cost models that pick intersection algorithms per query for web-search workloads. Adaptive execution takes the dynamic commitment further: Raman et al. [24] interleave RID-list intersection strategies based on observed selectivities precisely because static plans mispredict. The theory community established worst-case-optimal evaluation of union-intersection expressions [20, 23], and semijoin-style liveness reasoning over query plans dates to Yannakakis [1], recently revived as predicate transfer [69]. FROSTBIT occupies a point none of these target. Like the pre-execution optimizers it plans before evaluating, but from the *exact* per-container metadata of frozen operands rather than estimated statistics, and its output is stronger than an algorithm selection: the plan carries discharged obligations — capacity clamps, allocation-freeness, a closed-form memory bound, liveness masks — that estimate-driven planners cannot emit (their decisions may be wrong and must remain revisable) and that adaptive executors by construction do not have before execution. The trade is explicit: FROSTBIT forgoes runtime adaptivity on interior results (§9.5) in exchange for guarantees and an executor containing no decision code.

Compressed bitmap structures. The word-aligned RLE family (BBC [4], WAH [21], PLWAH [27], CONCISE [26]) optimizes space and bulk boolean throughput; sorting amplifies run lengths [28]. Roaring [43, 44, 54] introduced the two-level container design FROSTBIT adopts; FROSTBIT's frozen format transcodes losslessly to and from portable Roaring serialization at container granularity. Index-level designs — variant indexes [9], bitmap join indices [5], bit-sliced arithmetic [16], design-space studies [10] — and updatable structures (UpBit [47], TEB [61, 66], CUBIT [67]) are complementary: FROSTBIT deliberately targets immutable segments, which is the assumption that makes its static analysis sound. The bitmap-versus-inverted-list boundary is mapped in [52].

Container kernels. Adaptive and instance-optimal set intersection descends from Demaine et al. [15] and the Barbay line [22, 29], with engineering treatments for inverted indexes [30, 32]. The SIMD lineage comprises shuffle-based compare-all-pairs [34, 46], branch-reduced merge networks [39], QFilter [55], FESIA's segment prefilters [63], hierarchical partitioning (HERO [68]), and VP2INTERSECT alternatives [64]; worst-case-optimal join systems embed the same kernels relationally [40, 48]. FROSTBIT's kernels synthesize this line; CRoaring additionally ships per-pair dispatch heuristics (galloping thresholds, cardinality-based preallocation, deferred aggregation) [43, 54] that the Rust port lacks, a distinction our evaluation accounts for (§9.5).

Block skipping and scan pruning. Zone maps and small materialized aggregates prune blocks against predicate metadata [11, 49]; self-indexing inverted files skip within lists [7]; block-max indexes prune score-bounded blocks dynamically [33]. These techniques prune either along a single scan or during traversal. FROSTBIT's liveness analysis differs in operating *across* an arbitrary boolean expression statically, at Roaring's own container granularity, before any operand byte is read.

Memory discipline in query engines. Streaming and vectorized engines bound steady-state allocation operationally [19, 42, 65]; cache-aware cost modeling of operator behavior is classical [3,

[1]The term is borrowed from proof-carrying code [8] and is used in the same spirit as proof-carrying plans for AI planning [60]: the artifact transports obligations discharged before execution. FROSTBIT's plans carry compiler-derived bounds enforced by construction and checked by assertion and stress testing, not independently verifiable certificates.

18]; plan-time memory-requirement characterization has lineage in continuous queries over streams [17]; and representation selection for filter evaluation has a recent vectorized-execution treatment [62]. FROSTBIT's contribution here is the coupling: the memory bound is an artifact of the same analysis that proves capacity soundness, and serialization reuses the working arena (§4.4), which we have not seen documented for bitmap query execution.

Bitmaps in deployed systems. Signature and bitmap machinery appears throughout serving systems — signature files [2, 12], FastBit [25], BitFunnel [53], analytics engines [31, 38, 51, 57, 59, 70], graph search [37], and bit-parallel scan accelerators [35, 36, 41, 50, 58]. These document container structures and scan layouts; the expression-compilation layer described here is not, to our knowledge, described in any of them.

3 PRELIMINARIES

A FROSTBIT bitmap represents $S \subseteq [0, 2^{32})$, partitioned by key $\kappa(x) = \lfloor x/2^{16} \rfloor$ into containers over $[0, 2^{16})$, each stored as a sorted u16 array ($|c| \le A_{\max} = 4096$), a packed bitmap of $\mathsf{B} = 8192$ bytes, or a run list of $\rho(c) \le \rho_{\max} = 2047$ intervals:

$$\text{stored}(c) = \begin{cases} 2|c| & \text{array,} \\ \mathsf{B} & \text{bitmap,} \\ 2 + 4\rho(c) & \text{runs.} \end{cases}$$

The serialized form is a 16-byte header, a structure-of-arrays directory (8 bytes per container), and 64-byte-aligned payloads; buffers are validated once at the trust boundary and read zero-copy thereafter, including from memory maps.

A filter expression is a tree over frozen-bitmap leaves with operators $\wedge$, $\vee$ (arbitrary arity), and $\setminus$ (binary). Bare complement is not an operator; filter languages reach the engine with negation normalized into difference against an enclosing operand or an explicit universe bitmap. Same-operator edges are flattened at construction; an operator's *fan-in n* is its flattened operand count. $K(v)$ denotes the key set node v can emit; hats $\widehat{\cdot}$ denote plan-time upper bounds.

4 PLAN COMPILATION

Expression construction and analysis are fused: each combinator splices its children's programs into a flat post-order instruction sequence and computes the artifacts below bottom-up. A plan is specific to its operand set — leaf shapes are exact properties of particular frozen bitmaps — and may be evaluated against them arbitrarily many times; construction is cheap enough to sit on the query path and is included in every end-to-end measurement in §9.

4.1 Shapes

DEFINITION 1 (SHAPE). *The shape of node v is the key-sorted sequence $\{(k, \widehat{\text{card}}_k, \widehat{\rho}_k, \widehat{\tau}_k) : k \in K(v)\}$ of per-key bounds on output cardinality, run count, and representation under the kernel decision ladder.*

Leaf shapes are exact. Interior shapes follow operator semantics with saturating interval arithmetic ($\cap$/min for $\wedge$; $\cup$/Σ for $\vee$; left projection for $\setminus$) and are sound over-approximations of any execution. Because flattening changes arity, each spliced child contributes its *operand weight* to the parent's per-key fan-in bound $\widehat{n}_k$; weights make $\widehat{n}_k$ an exact upper bound on executor fan-in, which the promotion rule of §5 requires. (An early implementation that bounded fan-in by tree arity rather than flattened weight produced plans inconsistent with their kernels; the capacity stress suite of §4.2 detects this class of error, and the final design eliminates it by making the plan's clamp the sole authority for representation choice.)

4.2 Capacity clamps

Each operator receives an arena plan: one slot per key of its shape, with capacity

$$\text{cap}(k) = \begin{cases} \sigma(\widehat{\text{card}}_k) & \wedge \\ \max\left(\text{stored}_{\text{lhs}}(k),\ \sigma(\widehat{\text{card}}_k)\right) & \setminus \\ \max\left(2\widehat{\text{card}}_k,\ 2{+}4\widehat{\rho}_k,\ \max_i \text{stored}_i(k)\right) \text{ or } \mathsf{B} & \vee \end{cases}$$

where $\sigma(m) = 2m$ for $m \le A_{\max}$ and B otherwise, and the union clamp is B exactly when the plan assigns the key a bitmap or native-run accumulator (§5). One escape hatch trades slack for analysis cost: for tiny $\wedge$/$\setminus$ one-shots ($\le$16 KiB of input, $\le$32 driving keys), where §9 shows the analysis walk dominating, the planner substitutes the trivially sound clamp $\text{cap}(k) = \mathsf{B}$ over the driving input's keys and skips the capacity walk entirely.

PROPOSITION 1 (CAPACITY SOUNDNESS). *For every operator, key, and input consistent with the shapes, every intermediate state of the fold at key k occupies at most $\text{cap}(k)$ bytes.*

ARGUMENT. By case analysis on the accumulator ladder. For $\wedge$, the accumulator seeds from the per-key minimum-cardinality operand and only shrinks. One executor invariant carries the run case: a run encoding ($2{+}4\rho$ bytes) can exceed its array encoding when ρ approaches the cardinality, so run-form accumulators are permitted only inside B-clamped slots, where any $\rho \le \rho_{\max}$ container fits; under an array clamp the executor expands a run seed to array form at seeding. Every reachable state then fits $\sigma(\widehat{\text{card}}_k)$. For $\setminus$, untouched keys are verbatim copies; touched keys shrink from the left operand, and run-splitting states, bounded by $2 + 4(\rho_{\text{lhs}} + \mathit{items}_{\text{rhs}})$ bytes, arise only under a B clamp with conversion to bitmap forced before $\rho_{\max}$ is exceeded. For $\vee$, array accumulation is planned only when the summed bound fits an array, in which case every prefix fits; otherwise the clamp is B, which dominates bitmap, run, and conversion states. The executor never re-derives these decisions — the clamp *is* the decision — so plan/kernel divergence cannot occur. □

We state Proposition 1 and its relatives formally because the memory bound and the executor's structure depend on them, not as contributions to theory: the arguments are elementary. Their value is operational — debug builds assert every recorded container against its clamp, and an 11,600-case randomized stress suite exercises the obligations.

INVARIANT 1 (ALLOCATION-FREENESS). *The execution path contains no allocation, growth, or reallocation code.*

Table 1: Calibrated kernel constants (Apple M-series; medians over criterion runs).

c_b	per 8-lane merge block (compare-all-pairs)	1.45 ns
c_m	per element streamed through an array merge	0.85 ns
c_s	per value scattered into a bitmap	1.2 ns
c_0	bitmap clear + population count (fixed)	450 ns
c_p	bitmap membership probe (hoisted loop)	1.0 ns
c_g	galloping search probe (incl. mispredictions)	2.5 ns
c_x	bitmap→array result extraction, per value	0.25 ns

4.3 Working-memory bound

An operator with n_s slots occupies one arena comprising a reserved directory region $H(n_s) = \mathrm{align}_{64}(16 + 8n_s)$, the slot region, an optional mirror region for side-flipped folds (§6), and fixed scratch 2B:

$$A(P) = H(n_s) + (1{+}\delta_P)\,\mathrm{align}_{64}\Big(\textstyle\sum_k \mathrm{align}_8(\mathrm{cap}(k))\Big) + 2\mathsf{B}. \quad (1)$$

Since $\mathrm{cap} \le \mathsf{B}$, $A(P) \le H(n_s) + (1{+}\delta_P)n_s\mathsf{B} + 2\mathsf{B}$, and $\delta_P = 1$ only when $\sum_k \mathrm{cap}(k) \le F_{\max} = 64\,\mathrm{KiB}$, so the mirror contributes at most 64 KiB. For a tree, arenas are live along the evaluation stack, and peak working memory is the plan-time quantity $\max_s \sum_{v\in s} A(P_v)$ over stack states s.

4.4 In-place serialization

Because the directory region is reserved and slot capacities dominate payloads, serialization compacts payloads leftward within the arena and emits the arena's own buffer as the result.

Proposition 2 (Leftward compaction). *Writing outputs in key order at $d_i = \mathrm{base} + \sum_{j<i} \mathrm{size}_j$ (with per-type alignment) satisfies $d_i \le s_i$ for every source offset s_i; no unread byte is overwritten.*

Argument. $s_i = H{+}\sum_{j<i} \mathrm{align}_8(\mathrm{cap}_j){+}\Delta_i \ge \mathrm{base}{+}\sum_{j<i} \mathrm{size}_j = d_i$, as $\mathrm{base} \le H$ and $\mathrm{size}_j \le \mathrm{cap}_j$ pointwise; mirror-side sources lie beyond the primary region, and alignment rounding preserves the inequality because sources are at least as aligned as destinations. □

The operational content is one full pass of memory traffic removed relative to engines that copy results out of working memory. Profiling during development isolated this term directly: frostbit's 16-way difference fold was already faster than the baseline (185 vs. 190 ns) while the end-to-end operation was slower (206 vs. 190 ns); the gap was exactly the output copy, and eliminating it recovered the fold's advantage end-to-end (§9.3).

5 REPRESENTATION AND KERNEL SELECTION

Execution decisions are functions of plan-derived quantities with constants calibrated per microarchitecture (Table 1).

Sorted-array kernel tiers. For an array pair with $|A| \le |B|$: balanced pairs ($|B| \le 4|A|$) run a shuffle-merge (compare-all-pairs over rotations of both operands — five permutations per 8-lane block rather than seven — with table-driven output compaction) at cost $c_b\lceil(|A|{+}|B|)/8\rceil$; heavily skewed pairs run galloping binary search at $\approx c_g|A|\log_2|B|$, admitted only under the margin test $8\,|A|\log_2|B| < |B|$ so that the branch-heavy search must dominate the middle tier's linear window advance by a factor, not merely match it; intermediate skews run a galloping broadcast scan. Both boundaries were set empirically; replacing the margin test with a fixed ratio regressed five-way conjunctions by 26%.

Fan-in-aware promotion. A union key accumulating in array form re-streams its accumulator at every merge; with fan-in n and summed cardinality s,

$$C_{\mathrm{arr}}(n,s) \approx c_m\, s\Big(\tfrac{n+1}{2} - \tfrac{1}{n}\Big), \qquad C_{\mathrm{bmp}}(s) \approx c_0 + (c_s{+}c_x)\, s, \quad (2)$$

where c_x is the bitmap-to-array conversion of the finished accumulator, incurred precisely where promotion is contested ($s \le A_{\max}$, so the result re-materializes in array form). The crossover yields no promotion at $n \le 3$ for any $s \le A_{\max}$ and thresholds $s \gtrsim 970,480,200$ at $n = 4, 5, 8$, encoded as $\textsc{promote}(n,s) \equiv n \ge 4 \wedge s(n-3) > 1000$. Deployed Roaring implementations promote at fixed thresholds independent of fan-in: CRoaring's lazy aggregation defers conversion until an accumulator exceeds 1024 values, and the container specification converts at 4096 [43, 54]. Under the model no fixed threshold is correct across fan-ins, because the re-streamed volume grows with n; we measured both failure directions: a low fixed threshold (256) converts the OR-groups of low-fan-in conjunctions into bitmaps their parent consumes more slowly (+29% on that shape), and never promoting strands wide unions in array form (3.7× on 4-way unions; Table 4). At n=4 the derived threshold happens to land near CRoaring's 1024; the rules diverge everywhere else — never promoting at $n \le 3$ where a fixed 1024 would, and promoting at $s \approx 200$ at n=8 where a fixed 1024 waits 5× too long. Fan-in bounds come from the weighted shapes of §4.1.

Run containers fold natively (run×run intersection, union, and run-splitting difference on typed interval slices) inside B-clamped slots, converting to bitmaps only on proven overflow.

6 FOLD-ORDER SELECTION

An n-ary fold traverses keys × operands in one of two orders. *Key-major* order completes each key before the next: one accumulator remains cache-resident, but each operand stream is resumed once per key — nK prefetch restarts over K keys. *Partner-major* order streams each operand end-to-end against all accumulators: perfectly sequential input traffic, but the accumulator set is revisited n times. These are the container-algebra analogues of document-at-a-time and term-at-a-time postings traversal [6], and both move identical byte volumes; they differ only in arrangement relative to the cache hierarchy.

Profiling 16-way differences quantified the key-major penalty: the merge kernel sustains 1.45 ns per 8-lane block in isolation and 1.56 ns in a single-key control, but 2.17 ns inside a 32-key key-major fold (+50%), attributable to resuming 16 interleaved streams per key. Partner-major execution recovered the isolated rate — and dense unions then exhibited the inverse pathology: with per-key bitmap accumulators the accumulator set is $K\cdot\mathsf{B}$ (128 KiB at K=16), revisiting it each pass displaces L1, and key-major won by 12%.

The plan resolves the choice without estimation: the accumulator footprint $F(P) = \sum_k \mathrm{cap}(k)$ is already proven (Eq. 1). frostbit folds partner-major iff $F(P) \le F_{\max} = 64\,\mathrm{KiB}$ (half of L1D,

reserving room for one streaming operand and the mirror side), in which case the plan allocates the mirror region and array merges alternate slot sides, eliminating staging copies. Intersection is the measured exception: a key-count scaling control (16-way intersection over 1 vs. 32 keys: 695 ns $\rightarrow$ 21.6 ns, i.e. 31.1× for 32× the keys) shows no super-linear term — the accumulator collapses after the first operand and subsequent operands are searched, not streamed — and partner-major seeding would require clamping by the driver's cardinality rather than the per-key minimum, weakening Proposition 1. Intersection therefore remains key-major by measurement, difference and union dispatch on $F(P)$.

7 LIVENESS ANALYSIS

7.1 Static container pruning

Definition 2 (Liveness mask). $\mathcal{M}(T) = K(\mathrm{root}(T))$ *under the shape recurrences: intersection of child key sets at* $\wedge$*, union at* $\vee$*, left child at* $\setminus$.

Lemma 1 (Pruning soundness). *For any leaf* ℓ *and key* $k \notin \mathcal{M}(T)$*, deleting container* $c_k(\ell)$ *from the input changes no output of* T*. Moreover* $\mathcal{M}(T)$ *over-approximates the true surviving key set, so no contributing container is pruned.*

Argument. Induction on T: a key absent from $K(v)$ yields the empty container at v; empty containers are identities for $\vee$, annihilators for $\wedge$, and left-absorbing for $\setminus$, so absence propagates to the root. Executor key sets are subsets of shape key sets (§4.1), giving the over-approximation. □

The planner materializes $\mathcal{M}(T)$ as an 8 KiB bitset whenever $|K(\mathrm{root})| < \max_i |K(v_i)|$ — exactly the condition under which pruning is non-vacuous — and every leaf cursor skips dead keys in place. The streamed-bytes reduction is

$$\Delta = \sum_{\ell \in \mathrm{leaves}(T)} \sum_{k \in K(\ell) \setminus \mathcal{M}(T)} \mathrm{stored}\big(c_k(\ell)\big), \tag{3}$$

which for the selective-filter benchmark of §9 (one 4-key conjunct against four 256-key disjunction legs) removes 98% of input bytes and yields 7.1 ns versus 241 ns unpruned; enabling the automatic derivation improved the full 25k-tree corpus by 15.3% (criterion's change-detection test over 10 samples of 20 s, $p < 0.05$). Relative to runtime masking for flat intersections [71] and scan-side zonemap pruning [11, 49], the analysis here is static, applies to arbitrary operator trees, and carries a soundness argument tied to the plan's shapes.

7.2 Compiled short-circuits

The instruction sequence carries *guards* after each non-flattened $\wedge$-operand and each $\setminus$-minuend: an empty operand pops the partial state and branches past the fold, so sibling subtrees are never evaluated. An intersection whose planned key sets are disjoint is statically empty and skips its fold before reading any input byte; dynamically, every accumulator form exits its key the moment it empties. Guards use relative offsets and therefore survive splicing into enclosing plans. Within operators, intersection fuses population count into the word-level AND and abandons a key on emptiness; per-key folds order operands most-selective-first; partner-major passes skip dead slots. A filter containing a contradictory subexpression evaluates in 143 ns against 738 ns for the better baseline build, since the expensive sibling is never executed; we report this as a property of guard placement rather than a headline result, and note the baseline's operator API offers no way to express sibling short-circuiting, so the row compares expressiveness rather than kernels.

8 MEMORY DISCIPLINE

Four per-thread pools back execution: op arenas (Eq. 1), fold cursor/reference scratch, the evaluator's operand stack, and result buffers. Pooled buffers are stored empty and loaned at the caller's lifetime; intermediate arenas chain directly as operator inputs, and in-place serialization (Proposition 2) swaps the arena's buffer out as the result while a pooled result buffer rotates back. Steady-state evaluation therefore performs zero heap allocations — verified by sampling profiles in which the allocator does not appear — and first use on a thread allocates once, after which buffers circulate. The comparable baseline cost is visible in its own profiles: under the library's pairwise operators our workloads attribute roughly 6% of runtime to the allocator (its `MultiOps` aggregation path allocates less, via copy-on-write container borrowing, but is not allocation-free), before any tail-latency effects.

9 EVALUATION

Setup. Apple M4 Pro (aarch64/NEON); rustc 1.96.0 for FROSTBIT and the default baseline, rustc 1.98.0-nightly for the baseline's `simd` feature; criterion medians over 30 samples × 4 s per benchmark (corpus: 10 × 20 s), typical confidence half-widths under 2%. Baselines: Rust `roaring` 0.11.4 in both configurations — the default build (scalar array kernels; what stable-toolchain users run) and the nightly-only `simd` build — interleaved by a two-pass runner into one report, then a dedicated third pass re-measures the FROSTBIT cells alone under the same nightly build. The third pass exists because sub-microsecond FROSTBIT cells drift up to +70% inside the 20-minute interleaved passes — ambient allocator pressure from the alloc-heavy baseline benchmarks, which no short-context measurement reproduces — while baseline cells, whose own allocation traffic dominates that noise floor, show no such drift and keep their interleaved numbers. All n-ary baseline calls go through the library's `MultiOps` trait, its documented fast path for merging many bitmaps (lazy copy-on-write promotion, largest-first ordering); binary nodes use the pairwise operators. All FROSTBIT tree measurements include plan construction. Correctness: differential suites against Roaring (random trees, pruned trees, run containers, 10M-element round trips) and the 11.6k-case clamp stress; every configuration reported here passes all suites.

9.1 Operation sweep

Table 2 reports the full 2–16-way sweep over three container regimes; all 36 cells of all three systems come from one run of the protocol above. FROSTBIT wins 32 cells outright against both builds: every sorted-array cell (1.20–12.2× over default, 1.22–3.0× over SIMD), every intersection cell (1.04–5.8×), and every union cell (1.02–3.0×). The remaining four cells — difference at 2- and 4-way

in the dense and run regimes — sit at 0.94–1.00×, inside the measurement-noise band, and trade sides across repeated runs; the sweep contains no cell where either build beats FROSTBIT beyond noise.

The run-container regime did not start this way, and its repair is a two-part case study. The first part priced the analysis pass: run cells carry ~18-byte payloads, and an earlier version lost the binary cells outright (0.68× at binary run difference) because the planning walk alone was 282 ns of the 781 ns total while the fold was at parity with the baseline's entire operation — the cells isolated the cost of analysis with no data volume to amortize it against. Three measured responses (ablation rows in Table 4) recovered those cells: pooled plan buffers, a hybrid seed-driven plan walk with an inlined cursor fast path, and — decisively — capacity-analysis-free plans for tiny ∧/\ one-shots, which clamp every slot at B (sound: every fold state fits a bitmap slot, and the driving input's key set is a superset whose unclaimed slots serialize to nothing). The second part corrected a wrong diagnosis. The last holdout, 16-way run intersection, is an *annihilating* cell: the sweep's phase-shifted run windows sweep apart after roughly five operands, so the 16-way result is empty and the cell measures collapse handling, not merge throughput. The baseline's whole-bitmap accumulator makes every post-annihilation pass free — an empty accumulator has no containers to visit — while FROSTBIT's run fold, unlike its array and bitmap arms, lacked the empty-accumulator exit and ground through all $n-1$ passes. Restoring that exit, seeding the run fold from the minimum-cardinality container so annihilation arrives soonest, and skipping any fold whose planned key intersection is already empty flipped the cell (0.87× → 1.04×) and the partially-annihilating 8-way cell (to 2.26×); a statically disjoint intersection now costs the plan walk alone.

9.2 Filter-tree corpus

End-to-end evaluation uses 25,000 deterministic random filter trees (295,455 leaves; log-spread sizes 2–100 leaves; depths 2–15; per-tree shape profiles spanning deep conjunction chains, flat 100-way disjunctions, and difference-heavy composites; 100 trees pinned to both extremes), each built, planned, and materialized per iteration. FROSTBIT: **2.73 s** (9.2K trees/s); default baseline 7.14 s (2.61×); SIMD baseline 5.78 s (2.12×). Table 3 reports named shapes bracketing production patterns.

A per-tree audit — minimum-of-five timing of every corpus tree on both engines, structural feature extraction over trees where FROSTBIT trails, and root-cause analysis of the worst offenders — is part of the methodology: it directed two of the shipped mechanisms (the hoisted bitmap-membership loop, reducing the audit's offender count from 1,245 to 743 of 25,000; then automatic liveness derivation, the corpus-level −15.3%).

9.3 Ablations

Every mechanism was admitted under a keep-only-if-faster rule; Table 4 reports the ledger, including rejected designs, whose negative results delimit the design space. Three observations. First, the in-place-serialization entry illustrates layer-correct profiling: the fold outperformed the baseline while the operation lost, and the entire gap was the output copy. Second, branch discipline is empirical, not doctrinal: an adversarial-versus-predictable input sweep showed the merge advance branches cost nothing measurable; a branchless rewrite of the scalar union merge *regressed* 26% (predictable branches beat unconditional work); table-driven compaction of the intersection emit won 2.2× at dense overlap once gated to cost nothing at sparse overlap. Third, the fold-order controls (§6) scoped partner-major execution to the operators where the memory system rewards it.

9.4 Kernel parity

Because the claims concern the planning layer, we verified the kernels are not the explanation: per-block throughput against the baseline's SIMD kernels at equal work sits within a 0 to +16% band. A kernel edge of that size cannot account for the multiples in Tables 2–3, but it is not zero; the sharper attribution experiment — the baseline's kernels running under FROSTBIT's plans — is future work, and until then the end-to-end differences are attributable primarily, not exclusively, to planning, pruning, and memory discipline.

9.5 Limitations

Four limitations bound the present evaluation. (1) All measurements are from one aarch64 microarchitecture; the x86 (SSE) kernel ports compile and are algorithmically identical, but their performance and the transfer of Table 1's constants are unvalidated. (2) The library baseline family is the Rust Roaring library; CRoaring ships per-pair dispatch heuristics its Rust port lacks, so a CRoaring comparison with per-mechanism attribution is required before the library multiples can be read as pure planning-layer gains. The incumbent comparison of §10 partially closes this — same kernel class, decisions re-derived per evaluation — but CRoaring remains unmeasured. (3) Operands and trees are synthetic (deterministic generators); standard Roaring real-data corpora and production filter traces remain future work. (4) Interior-node bounds are sound but loose, and static dispatch on loose bounds forgoes the adaptivity of dynamic schemes [24]; quantifying bound tightness and mis-dispatch frequency against an oracle is open. The evaluation is single-threaded; the per-thread pool design has not been characterized under concurrency.

10 DEPLOYMENT AND THE INCUMBENT SUBSTRATE

FROSTBIT is the successor to the filtering substrate of Exa, a commercial web-scale search engine, where every boolean filter predicate is evaluated against frozen per-segment bitmaps served from memory maps. Plans are per-(expression, segment) — the analysis runs against each segment's exact leaf metadata, which is what makes its guarantees possible, and its cost sits on the query path exactly as in every measurement of §9.

Head-to-head against the incumbent. The deployed substrate ("FROZ-v2"), from which FROSTBIT was extracted, is the sharpest available test of this paper's thesis: it shares the design lineage — the same container algebra, SIMD kernel class, arena execution, in-place serialization, and a runtime hole-punching mode — but re-derives its decisions during each evaluation instead of carrying them in a plan. We ran both engines on the identical workloads of §9 (same generators and seeds; outputs verified equal on every

Table 2: Full operation sweep, medians in µs: FROSTBIT / Roaring default / Roaring `simd`, n-ary baseline calls via `MultiOps`. Sparse = 32-key arrays (800/key); dense = 16-key bitmaps (5000/key); runs = 16-key run containers.

op	regime	2-way	4-way	8-way	16-way
∧	sparse	10.4 / 59.9 / 17.3	14.1 / 69.5 / 28.6	16.7 / 58.8 / 28.6	20.3 / 64.6 / 28.5
∧	dense	15.6 / 16.6 / 16.7	13.7 / 20.1 / 19.9	12.6 / 20.4 / 20.1	14.7 / 20.4 / 20.3
∧	runs	0.66 / 0.87 / 0.86	1.20 / 1.90 / 2.00	1.20 / 2.70 / 2.60	2.60 / 2.70 / 2.70
∨	sparse	46.3 / 85.7 / 83.1	54.6 / 164.9 / 163.0	116.5 / 139.3 / 142.7	229.3 / 282.7 / 290.4
∨	dense	5.00 / 5.10 / 5.20	9.70 / 11.4 / 11.5	20.3 / 23.8 / 24.1	42.0 / 48.7 / 49.3
∨	runs	0.81 / 0.93 / 0.91	1.60 / 2.20 / 2.20	3.10 / 4.80 / 4.80	5.70 / 7.40 / 7.40
\	sparse	11.6 / 52.7 / 17.5	31.7 / 336.9 / 50.0	74.1 / 905.7 / 124.9	186.3 / 2020 / 318.7
\	dense	5.70 / 5.40 / 5.50	51.2 / 50.1 / 51.0	54.4 / 131.7 / 137.7	66.4 / 241.0 / 248.3
\	runs	0.59 / 0.55 / 0.58	1.00 / 1.00 / 1.00	1.90 / 2.10 / 2.10	3.40 / 3.70 / 3.80

Table 3: Filter-tree shapes, medians (µs); plan construction included. On *selective* the baseline's pairwise operators beat its `MultiOps` path (≈0.85 ms in a pairwise-baseline run); FROSTBIT's pruning multiple exceeds 100× against either.

shape	FROSTBIT	R	R^{simd}
conj5 (nested ∧, flattens 5-way)	**34.9**	265	70.9
filter (base ∧ OR-group ∧ ¬lang)	**17.8**	40.6	41.7
dnf (∨ of ∧-groups)	**52.5**	614	112
cnf3 (∧ of ∨-groups)	**74.2**	105	94.0
selective (statically pruned)	**7.6**	1450	1520
contradiction (guarded)	**0.147**	738	747
25k corpus (seconds)	**2.73**	7.14	5.78

Table 4: Ablation ledger (kept above the line, rejected below). Entries are measurements at the time each mechanism landed; the surrounding system has since moved, so endpoints differ from the final-system tables.

mechanism	effect
balanced shuffle-merge (replacing scan)	dnf 130 → 57 *ns*
register-resident merge rewrite	2-way ∧: 49 → 24.8 *ns*
fan-in-aware promotion	4-way ∨: 203 → 54.9 *ns*
partner-major \ (footprint-gated)	8-way sparse: 85 → 72 *ns*
in-place serialization	dense 2-way ∨: 7.2 → 5.1 *ns*
hoisted bitmap membership loop	audit offenders 1245 → 743
automatic liveness derivation	corpus 3.10 → 2.63 s
gated branchless ∧ emit	dense-hit emit 2.07 → 0.95 *ns*
pooled plan buffers (slots + cursors)	\ runs/2: 781 → 692 ns
inlined `advance_to` + hybrid plan walk	∧ sp/16: 20.6 → 19.9 *ns*
trivial B-plans, tiny ∧/\ one-shots	\ runs/2: 692 → 607 ns
monomorphic run fold + fused seed scan	∧ runs/16: 3.5 → 3.3 *ns*
run-fold empty exit + min-card seed	∧ runs/16: 3.1 → 2.6 *ns*
eager bitmap intermediates ($s > 256$)	cnf3 89 → 115 *ns*
branchless scalar ∨ merge	72 → 91 *ns*
deferred vector-mask accumulation	\: 83 → 97 *ns*
window pre-clipping before merges	redundant with galloping
partner-major ∧	no gain (linear-scaling control)

op/regime, every named shape, and every 64th corpus tree), with the incumbent's production telemetry layer stubbed to no-ops —

Table 5: FROSTBIT vs. the deployed incumbent (FROZ-v2), identical workloads, medians; incumbent shown plain and with its runtime hole-punching enabled.

shape	FROSTBIT	v2	$v2^{punch}$
25k corpus (s)	**2.62**	2.84	2.71
selective (µs)	**8.5**	265	35.4
filter (µs)	**17.6**	33.2	23.7
dnf (µs)	**52.0**	182	184
conj5 (µs)	**34.2**	42.3	44.0
cnf3 (µs)	**73.0**	79.0	81.5

which only flatters it, since production pays per-operation metric costs FROSTBIT does not have. Table 5 reports the result.

On the flat sweep FROSTBIT wins 34 of 36 cells: sparse difference by 5.4–12.3× (the incumbent folds its n-ary difference pairwise), sparse intersection by 1.9–3.3×, dense intersection up to 3.0×, run cells by 1.3–1.6× (the incumbent's run fold also lacks the empty-accumulator exit of §9), and unions mostly by 1.2–2.2×; the incumbent keeps binary sparse union (0.85× — its union kernel is markedly stronger than the Rust Roaring library's) and ties 16-way sparse union. On eight random shape-class trees FROSTBIT wins five, ties one, and loses two (worst 0.50× on one 49-leaf shape, an unresolved offender). The aggregate corpus gain over this incumbent is deliberately reported as what it is — +8% plain, +3% against its punched mode — because that is the honest shape of the result: against an equally-tuned engine of the same lineage, static planning's value concentrates not in the aggregate but in the tails it removes (unpunched selective filters, 31×; union promotion on DNF shapes, 3.6×; every sparse-difference shape, 5–12×) and in the guarantees — allocation-freeness, the memory bound, static liveness — that per-evaluation re-analysis cannot emit. A fleet-scale production characterization (tail-latency distributions, filter-shape statistics) remains beyond this paper's scope; the incumbent comparison above was measured on the same hardware and protocol as every other number reported here.

11 CONCLUSION

For boolean filtering over compressed bitmaps in the frozen-segment serving regime, the optimization surface that prior work resolves dynamically can be discharged entirely at plan time, and doing so

yields guarantees dynamic schemes cannot offer: allocation-free execution under a closed-form memory bound, sound static pruning at container granularity across arbitrary expression trees, and fold orders selected from proven footprints. On kernels measured at parity with the baseline's, these plan-time guarantees account for the bulk of outright wins in 32 of 36 sweep cells — sorted-array operations by 1.2–12.2× — with the remaining four inside measurement noise and none lost, and 2.1–2.6× end-to-end against the standard Rust Roaring library's documented fast paths. The system, its differential and stress suites, the 25k-tree corpus, and the audit tooling are open source.[2]

REFERENCES

[1] M. Yannakakis. Algorithms for acyclic database schemes. *VLDB*, 1981.

[2] C. Faloutsos, S. Christodoulakis. Signature files: an access method for documents and its analytical performance evaluation. *ACM TOIS* 2(4), 1984.

[3] M. E. Wolf, M. S. Lam. A data locality optimizing algorithm. *PLDI*, 1991.

[4] G. Antoshenkov. Byte-aligned bitmap compression. *DCC*, 1995.

[5] P. O'Neil, G. Graefe. Multi-table joins through bitmapped join indices. *SIGMOD Record* 24(3), 1995.

[6] H. Turtle, J. Flood. Query evaluation: strategies and optimizations. *Information Processing & Management* 31(6), 1995.

[7] A. Moffat, J. Zobel. Self-indexing inverted files for fast text retrieval. *ACM TOIS* 14(4), 1996.

[8] G. C. Necula. Proof-carrying code. *POPL*, 1997.

[9] P. O'Neil, D. Quass. Improved query performance with variant indexes. *SIGMOD*, 1997.

[10] C.-Y. Chan, Y. E. Ioannidis. Bitmap index design and evaluation. *SIGMOD*, 1998.

[11] G. Moerkotte. Small materialized aggregates: a light weight index structure for data warehousing. *VLDB*, 1998.

[12] J. Zobel, A. Moffat, K. Ramamohanarao. Inverted files versus signature files for text indexing. *ACM TODS* 23(4), 1998.

[13] T. Johnson. Performance measurements of compressed bitmap indices. *VLDB*, 1999.

[14] S. Amer-Yahia, T. Johnson. Optimizing queries on compressed bitmaps. *VLDB*, 2000.

[15] E. D. Demaine, A. López-Ortiz, J. I. Munro. Adaptive set intersections, unions, and differences. *SODA*, 2000.

[16] D. Rinfret, P. O'Neil, E. O'Neil. Bit-sliced index arithmetic. *SIGMOD*, 2001.

[17] A. Arasu, B. Babcock, S. Babu, J. McAlister, J. Widom. Characterizing memory requirements for queries over continuous data streams. *PODS*, 2002.

[18] S. Manegold, P. Boncz, M. Kersten. Generic database cost models for hierarchical memory systems. *VLDB*, 2002.

[19] P. Boncz, M. Zukowski, N. Nes. MonetDB/X100: hyper-pipelining query execution. *CIDR*, 2005.

[20] E. Chiniforooshan, A. Farzan, M. Mirzazadeh. Worst case optimal union-intersection expression evaluation. *ICALP*, 2005.

[21] K. Wu, E. J. Otoo, A. Shoshani. Optimizing bitmap indices with efficient compression. *ACM TODS* 31(1), 2006.

[22] J. Barbay, A. López-Ortiz, T. Lu. Faster adaptive set intersections for text searching. *WEA*, 2006.

[23] P. Bille, A. Pagh, R. Pagh. Fast evaluation of union-intersection expressions. *ISAAC*, 2007.

[24] V. Raman, L. Qiao, W. Han, I. Narang, Y. Chen, K.-H. Yang, F.-L. Ling. Lazy, adaptive RID-list intersection, and its application to index anding. *SIGMOD*, 2007.

[25] K. Wu et al. FastBit: interactively searching massive data. *J. Phys.: Conf. Ser.* 180, 2009.

[26] A. Colantonio, R. Di Pietro. CONCISE: compressed 'n' composable integer set. *Inf. Process. Lett.* 110(16), 2010.

[27] F. Deliège, T. B. Pedersen. Position list word aligned hybrid: optimizing space and performance for compressed bitmaps. *EDBT*, 2010.

[28] D. Lemire, O. Kaser, K. Aouiche. Sorting improves word-aligned bitmap indexes. *Data & Knowl. Eng.* 69(1), 2010.

[29] J. Barbay, A. López-Ortiz, T. Lu, A. Salinger. An experimental investigation of set intersection algorithms for text searching. *ACM JEA* 14, 2010.

[30] J. S. Culpepper, A. Moffat. Efficient set intersection for inverted indexing. *ACM TOIS* 29(1), 2010.

[31] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, T. Vassilakis. Dremel: interactive analysis of web-scale datasets. *PVLDB* 3(1), 2010.

[32] B. Ding, A. C. König. Fast set intersection in memory. *PVLDB* 4(4), 2011.

[33] S. Ding, T. Suel. Faster top-k document retrieval using block-max indexes. *SIGIR*, 2011.

[34] B. Schlegel, T. Willhalm, W. Lehner. Fast sorted-set intersection using SIMD instructions. *ADMS@VLDB*, 2011.

[35] Y. Li, J. M. Patel. BitWeaving: fast scans for main memory data processing. *SIGMOD*, 2013.

[36] L. Sidirourgos, M. Kersten. Column imprints: a secondary index structure. *SIGMOD*, 2013.

[37] M. Curtiss et al. Unicorn: a system for searching the social graph. *PVLDB* 6(11), 2013.

[38] F. Yang, E. Tschetter, X. Léauté, N. Ray, G. Merlino, D. Ganguli. Druid: a real-time analytical data store. *SIGMOD*, 2014.

[39] H. Inoue, M. Ohara, K. Taura. Faster set intersection with SIMD instructions by reducing branch mispredictions. *PVLDB* 8(3), 2014.

[40] T. L. Veldhuizen. Leapfrog Triejoin: a simple, worst-case optimal join algorithm. *ICDT*, 2014.

[41] Z. Feng, E. Lo, B. Kao, W. Xu. ByteSlice: pushing the envelop of main memory data processing with a new storage layout. *SIGMOD*, 2015.

[42] B. Chandramouli, J. Goldstein, M. Barnett, R. DeLine, D. Fisher, J. C. Platt, J. F. Terwilliger, J. Wernsing. Trill: a high-performance incremental query processor for diverse analytics. *PVLDB* 8(4), 2015.

[43] S. Chambi, D. Lemire, O. Kaser, R. Godin. Better bitmap performance with Roaring bitmaps. *Softw. Pract. Exp.* 46(5), 2016.

[44] D. Lemire, G. Ssi-Yan-Kai, O. Kaser. Consistently faster and smaller compressed bitmaps with Roaring. *Softw. Pract. Exp.* 46(11), 2016.

[45] O. Kaser, D. Lemire. Compressed bitmap indexes: beyond unions and intersections. *Softw. Pract. Exp.* 46(2), 2016.

[46] D. Lemire, L. Boytsov, N. Kurz. SIMD compression and the intersection of sorted integers. *Softw. Pract. Exp.* 46(6), 2016.

[47] M. Athanassoulis, Z. Yan, S. Idreos. UpBit: scalable in-memory updatable bitmap indexing. *SIGMOD*, 2016.

[48] C. R. Aberger, A. Lamb, S. Tu, A. Nötzli, K. Olukotun, C. Ré. EmptyHeaded: a relational engine for graph processing. *SIGMOD*, 2016.

[49] B. Dageville et al. The Snowflake elastic data warehouse. *SIGMOD*, 2016.

[50] H. Lang, T. Mühlbauer, F. Funke, P. Boncz, T. Neumann, A. Kemper. Data Blocks: hybrid OLTP and OLAP on compressed storage. *SIGMOD*, 2016.

[51] S. Chambi, D. Lemire, R. Godin, K. Boukhalfa, C. Allen, F. Yang. Optimizing Druid with Roaring bitmaps. *IDEAS*, 2016.

[2] https://github.com/WilliXL/frostbit

[52] J. Wang, C. Lin, Y. Papakonstantinou, S. Swanson. An experimental study of bitmap compression vs. inverted list compression. *SIGMOD*, 2017.

[53] B. Goodwin, M. Hopcroft, D. Luu, A. Clemmer, M. Curmei, S. Elnikety, Y. He. BitFunnel: revisiting signatures for search. *SIGIR*, 2017.

[54] D. Lemire, O. Kaser, N. Kurz, L. Deri, C. O'Hara, F. Saint-Jacques, G. Ssi-Yan-Kai. Roaring bitmaps: implementation of an optimized software library. *Softw. Pract. Exp.* 48(4), 2018.

[55] S. Han, L. Zou, J. X. Yu. Speeding up set intersections in graph algorithms using SIMD instructions. *SIGMOD*, 2018.

[56] S. Kim, T. Lee, S.-w. Hwang, S. Elnikety. List intersection for web search: algorithms, cost models, and optimizations. *PVLDB* 12(1), 2018.

[57] J.-F. Im et al. Pinot: realtime OLAP for 530 million users. *SIGMOD*, 2018.

[58] B. Hentschel, M. S. Kester, S. Idreos. Column Sketches: a scan accelerator for rapid and robust predicate evaluation. *SIGMOD*, 2018.

[59] B. Chattopadhyay et al. Procella: unifying serving and analytical data at YouTube. *PVLDB* 12(12), 2019.

[60] C. Schwaab, E. Komendantskaya, A. Hill, F. Farka, R. P. A. Petrick, J. Wells, K. Hammond. Proof-carrying plans. *PADL*, 2019.

[61] H. Lang, A. Beischl, V. Leis, P. Boncz, T. Neumann, A. Kemper. Tree-encoded bitmaps. *SIGMOD*, 2020.

[62] A. Ngom, P. Menon, M. Butrovich, L. Ma, W. S. Lim, T. C. Mowry, A. Pavlo. Filter representation in vectorized query execution. *DaMoN*, 2021.

[63] J. Zhang, Y. Lu, D. G. Spampinato, F. Franchetti. FESIA: a fast and SIMD-efficient set intersection approach on modern CPUs. *ICDE*, 2020.

[64] G. Díez-Cañas. Faster-than-native alternatives for x86 VP2INTERSECT instructions. arXiv:2112.06342, 2021.

[65] A. Behm et al. Photon: a fast query engine for lakehouse systems. *SIGMOD*, 2022.

[66] M. P. Saputra, E. Tzirita Zacharatou et al. In-place updates in tree-encoded bitmaps. *DaMoN*, 2022.

[67] J. Wang, M. Athanassoulis. CUBIT: concurrent updatable bitmap indexing. *PVLDB* 17, 2024.

[68] B. Yang, W. Zheng, X. Lian, Y. Cai, X. S. Wang. HERO: a hierarchical set partitioning and join framework for speeding up the set intersection over graphs. *Proc. ACM Manag. Data (SIGMOD)*, 2024.

[69] Y. Yang, H. Zhao, X. Yu, P. Koutris. Predicate transfer: efficient pre-filtering on multi-join queries. *CIDR*, 2024.

[70] R. Schulze, T. Schreiber, I. Yatsishin, R. Dahimene, A. Milovidov. ClickHouse — lightning fast analytics for everyone. *PVLDB* 17(12), 2024.

[71] The RoaringBitmap Java library, `org.roaringbitmap` (`workShyAnd` aggregation). https://github.com/RoaringBitmap/RoaringBitmap, accessed 2026.